

Juju Charms offer an amazing experience for private and public cloud deployments, including bare metal with Metal as a Service (MAAS), OpenStack, AWS, Azure, Google Cloud, and more.

Architecture

Figure 4 illustrates the architecture of Juju Charms.

Juju's environment is bootstrapped locally or on top of an IaaS, creating the state service machine, which manages other machines.

The state service fetches the Charm that is to be deployed on the admin's request from the Charms Store or the local repository (if passed by *args*). The state service creates a machine (VM or container, depending on the IaaS) that contains an agent. This agent communicates with the state service, passing information to it. The Charm code runs in one unit, which can be executed in one or more machines. Admins can manage the environment via the Juju CLI or Juju GUI.

Features

The features of Juju Charms include:

- Easy deployment, modelling, scalability and management of services.
- An easy and interactive GUI to manage all services; overall, it is easy to install, use and configure.
- Service configuration as per user requirements.
- Avoids dependency problems in different machines while running services where Juju is installed.
- Can create and contribute a Charm with the help of the community.

Official website: <https://jujucharms.com/>

Latest version: 2.3

Puppet

Puppet is an open source software configuration tool. It is a declarative language for write-once-deploy-many packages for on-demand OpenStack configuration and version control.

Puppet provides full support for

OpenStack, and the latter's team has even devoted a page for Puppet's support. The modules described on this page are created by the OpenStack community for Puppet and, as such, reside on OpenStack's own Git repository system. Puppet is deployed in a client/server setup, or in serverless mode, where clients periodically poll the server for the desired state and send back status reports to the server (master). Puppet can provision, upgrade and manage nodes throughout their life cycle. Puppet is written in Ruby language, provides custom DSL for writing manifests and makes use of ERB for templates. It has a Web UI and reporting tools.

Puppet Enterprise allows for real-time control of managed nodes using prebuilt modules and the Cookbooks present on the master servers. The reporting tools are well developed, providing details on how agents are behaving and what changes have been made.

Components

The following are the key components of Puppet.

- **Puppet resources:** These comprise the key components for modelling any particular machine and have their own implementation model. They are used by Puppet to get a particular resource in the desired state.
- **Providers:** Providers never do anything on their own; all of their action is triggered through an associated resource.
- **Manifest:** This is a collection of resources inside the function or classes, which helps to configure any target system and makes use of the

Ruby language to configure a system.

- **Modules:** These act as building blocks of Puppet and consist of resources, files, templates, etc.
- **Templates:** These use Ruby expressions to define the customised content and variable input. They can be used by developers to create custom content.
- **Static files:** These are general files that perform specific tasks.

Architecture

Figure 5 illustrates the architecture of Puppet, which comprises the following.

- **Master:** This handles all configuration related tasks. It applies the configuration to nodes through the Puppet agent.
- **Agents:** These are machines that make up the entire network and are managed by the Master. All agents run the Puppet Agent Daemon service.
- **Config repository:** All the configurations of nodes and servers are saved and acquired from the config repository.
- **Facts:** These are details regarding the nodes or master machine, and are used to analyse the current status of any node.
- **Catalogue:** All the manifest files or configurations written in Puppet are first converted to a compiled format called a catalogue and later this is applied on the target machine.

Features

The features of Puppet include the following:

- It enables deployment and configuration of OpenStack components, like Nova, Keystone, Glance and Swift.
- Facilitates real-time control of managed nodes using prebuilt modules and Cookbooks present on master servers.
- Puppet supports idempotency, which makes it unique. Similar to Chef, in Puppet, one can safely run the same configuration multiple times on the same machine. In this

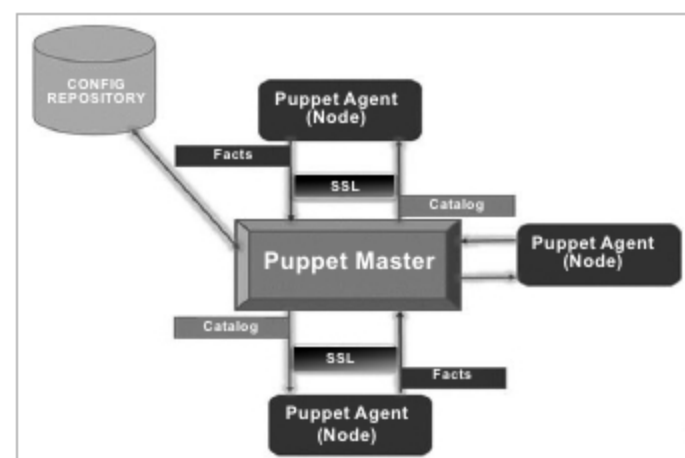


Figure 5: Puppet architecture